

Sprawozdanie z projektu zaliczeniowego przedmiotu Systemy Operacyjne Czasu Rzeczywistego

Symulator operacji windy budynkowej w języku Java

Autorzy:

Jan Machowski, 325492
Bartłomiej Modzolewski, 331719
Karolina Wiśniewska, 331757

Semestr: 26L
Język: Java 17+
Środowisko GUI Swing

0 Spis treści

1	Wstęp	2
2	Struktura projektu	3
3	Opis klas	3
3.1	Klasa <code>Entity</code>	3
3.1.1	Opis klasy	3
3.1.2	Opis stanów <code>Entity.State</code>	3
3.1.3	Najważniejsze metody:	4
3.2	Klasa <code>Floor</code>	4
3.2.1	Opis klasy	4
3.2.2	Najważniejsze metody:	4
3.3	Klasa <code>Elevator</code>	5
3.3.1	Opis klasy	5
3.3.2	Opis stanów <code>Elevator.State</code>	5
3.3.3	Kolejka <code>destinationQueue</code>	6
3.3.4	Najważniejsze metody:	6
3.4	Klasa <code>Division</code>	7
3.5	Klasa <code>ElevatorGUI</code>	8
4	Uruchomienie projektu	9
4.1	Wymagania	9
4.2	Kompilacja i uruchomienie z wiersza poleceń	9
4.3	Uruchomienie z IDE	9
4.4	Konfiguracja parametrów w GUI	9
5	Architektura i przepływ danych	9
6	Podsumowanie	10

1 Wstęp

Niniejszy dokument stanowi sprawozdanie z projektu zaliczeniowego w ramach zajęć projektowych przedmiotu Systemy Operacyjne Czasu Rzeczywistego, odbywających się w semestrze letnim roku 2026. Projekt polegał na stworzeniu aplikacji symulującej działanie przykładowego, wybranego przez zespół, systemu operacyjnego czasu rzeczywistego, oraz przedstawienie jego działania.

Wybrany przez zespół temat aplikacji to system obsługujący działanie windy budynku. Projekt zakładał stworzenie oprogramowania obsługującego, oraz środowiska symulującego działanie windy, wraz z graficznym interfejsem użytkownika.

Kluczowymi aspektami do rozwiązania w celu ukończenia projektu były:

1. Zapewnienie ciągłego działania systemu windy, niezależnie od liczby oczekujących osób;
2. Wykorzystanie odpowiedniego algorytmu kolejkowania, w celu realistycznego odtworzenia działania systemu;
3. Umożliwienie modyfikacji podstawowych parametrów windy, takich jak:
 - Liczba pięter, piętro początkowe i końcowe;
 - Prędkość przejazdu windy między piętrami;
 - Czas oczekiwania na obsługę wejścia / wyjścia pasażerów;
 - Maksymalna liczba osobników w windzie;
 - Piętro początkowe windy.
4. Stworzenie odpowiedniego środowiska symulacyjnego dla codziennego działania systemu, w co wchodzi m.in. dodawanie pasażerów na obsługiwanych piętrach.
5. Wykonanie funkcjonalnego i intuicyjnego interfejsu graficznego użytkownika w celu obsługi symulacji.

Najistotniejszym elementem całego projektu jest stworzenie wykorzystanie algorytmu kolejkowania. Jak można się domyśleć, winda nie może przykładowo spontanicznie zatrzymać się na piętrze, z którego natychmiast wcześniej zostało zarejestrowane wezwanie, oraz nie powinna ignorować wezwania które zostało wykonane znaczny czas wcześniej.

Kolejnym istotnym elementem było zapewnienie funkcjonalności symulacyjnych windy - zarówno manualnych, jak i automatycznych. W to wchodzi m.in. ręczne oraz losowe tworzenie pasażerów czekających na odpowiednich piętrach, obsługa ich zapytań tak, żeby jak najwięcej osób dostało się na swoje wymagane piętra w odpowiednim czasie a także unikanie przypadku, w którym winda kulkukrotnie przejeżdża przez piętro bez zatrzymania się na nim.

Wynikiem projektu jest załączony w katalogu `src/` projekt **Elevator**, który jest skończoną implementacją symulacji systemu operacyjnego czasu rzeczywistego. Ni-

niejsze sprawozdanie opisywać będzie strukturę, działanie, wykorzystanie oraz kluczowe aspekty napotkane podczas tworzenia projektu.

2 Struktura projektu

Projekt podzielony jest na dwa pakiety:

- `elevator.model` - logika backend'owa symulacji - model danych, klas wykorzystanych w projekcie i algorytmy sterowania windą i obsługi przypadków;
- GUI - graficzny interfejs użytkownika oparty na bibliotece Swing.

Poniższa tabela przedstawia klasy stworzone w ramach implementacji projektu wraz z ich przynależnością:

Klasa	Pakiet	Rola
Entity	<code>elevator.model</code>	Reprezentacja pasażera
Floor	<code>elevator.model</code>	Piętro budynku
Elevator	<code>elevator.model</code>	Winda
Division	<code>elevator.model</code>	Pion budynku (zbiór n pięter i jednej windy)
ElevatorGUI	GUI	Okno aplikacji Swing

3 Opis klas

3.1 Klasa Entity

3.1.1 Opis klasy

Klasa reprezentująca pojedynczego pasażera (osobnika) w symulacji. Każda instancja posiada 4 istotne informacje:

1. **Unikalny identyfikator**, nadawany automatycznie za pomocą licznika atomowego (`AtomicInteger`);
2. `originFloor` - piętro, z którego pochodzi i na którym oczekuje na windę pasażer. Jest wstawiane w kolejkę `destinationQueue` gdy pasażer się pojawia, oraz usuwane z niej gdy pasażer wsiada do windy;
3. `destinationFloor` - piętro, na którym pasażer chce wysiąść. Jest wstawiane w kolejkę `destinationQueue` gdy pasażer wsiada do windy oraz usuwane gdy pasażer wysiada na wybranej windzie;
4. **Jeden z trzech stanów** `Entity.State`, opisanych poniżej.

3.1.2 Opis stanów Entity.State

Każdy osobnik klasy `Entity` posiada jeden z trzech stanów `Entity.State` opisujących aktualne zadanie pasażera:

- `WAITING_ON_FLOOR` - pasażer oczekuje na windę na piętrze początkowym;

- `IN_ELEVATOR` - pasażer znajduje się w kabinie windy, jedzie na piętro docelowe;
- `ARRIVED` - pasażer dotarł do celu i opuścił windę. Po opuszczeniu windy, obiekt klasy `Entity` jest usuwany.

Obiekty klasy `Entity` zawarte są we wszystkich trzech klasach funkcjonalnych, tj. `Elevator`, `Floor` i `Division`.

3.1.3 Najważniejsze metody:

`boardElevator()`

Zmienia stan pasażera z `WAITING_ON_FLOOR` na `IN_ELEVATOR`. Wyrzuca `IllegalStateException`, gdy pasażer nie oczekuje na piętrze;

`exitElevator()`

Zmienia stan z `IN_ELEVATOR` na `ARRIVED`. Analogicznie zabezpieczona walidacją aktualnego stanu pasażera, co `boardElevator`;

`getId()`, `getOriginFloor()`, `getDestinationFloor()`, `getState()`

Funkcje zwracające wartości pól niezmiennych - identyfikatora, piętra startowego, docelowego oraz aktualnego stanu.

3.2 Klasa `Floor`

3.2.1 Opis klasy

Klasa reprezentująca jedno piętro budynku, do którego dostępna jest winda. Implementuje `Comparable<Floor>` na podstawie numeru piętra, dzięki czemu piętra można porównywać i sortować (używane w `Division` do wyznaczania `minFloor` i `maxFloor`). Klasa ta zawiera:

1. `floorNumber` - numer piętra w kondygnacji;
2. `waitingEntities` - lista pasażerów oczekujących na danym piętrze. Gdy winda zatrzyma się na danym piętrze, oraz nie jest w pełni zapełniona, obiekty osobników z tej listy trafiają do listy `occupants` klasy `Elevator`;

3.2.2 Najważniejsze metody:

`spawnEntity(int destinationFloor)`

Generuje nowy obiekt klasy `Entity` - pasażera na piętrze z podanym piętrem docelowym, dodaje go do listy oczekujących i zwraca referencję. Wyrzuca `IllegalArgumentException` przy próbie stworzenia pasażera z piętrem docelowym równym bieżącemu;

`entityEntersElevator(Entity entity)`

Usuwa pasażera z listy oczekujących i wywołuje `entity.boardElevator()` na danym pasażerze. Wyrzuca `IllegalArgumentException`, gdy pasażer nie należy do tej listy.

`entityExitsElevator(Entity entity)`

Wywołuje `entity.exitElevator()` - pasażer dotarł na miejsce i "znika" z okolicy windy.

`hasWaitingEntities(), getWaitingCount(), getWaitingEntities()`

Metody informacyjne o stanie piętra; `getWaitingEntities()` zwraca niemodyfikowalny widok listy.

3.3 Klasa Elevator

3.3.1 Opis klasy

Klasa `Elevator` jest główną klasą logiki symulacji. Zarządza windą znajdującą się w kondygnacji `Division`: przechowuje informacje o jej aktualnej pozycji, listę pasażerów wewnątrz kabiny, kolejkę pięter docelowych oraz stan ruchu.

Winda porusza się synchronicznie w dedykowanym wątku, blokując go podczas symulacji czasu przejazdu i postoju (`Thread.sleep`).

Klasa ta zawiera następujące informacje:

1. `destinationQueue` - kolejka priorytetowa klasy `Elevator`. Jest to główna część aplikacji, na której opiera się cała interpretacja systemu operacyjnego czasu rzeczywistego. Do tej kolejki dodawane są piętra, które winda ma odwiedzić, w kolejności ich istotności - opisane w sekcji 3.3.3.
2. `currentState` - aktualny stan windy, wybrany jako jeden spośród dostępnych stanów `Elevator.State` - opisane w sekcji 3.3.2;
3. `currentFloor` - aktualne piętro (numer piętra, nie obiekt klasy `Floor`), na którym znajduje się winda;
4. `speedMs` - prędkość, z jaką winda porusza się między piętrami. W praktyce jest to czas oczekiwania między "przeskoczeniem" windy między piętrami;
5. `IOwaitMs` - czas, jaki winda czeka podczas wchodzenia / wychodzenia pasażerów. Symuluje to m.in. otwarcie drzwi i wymianę pasażerów;
6. `maxCapacity` - maksymalna liczba pasażerów windy dozwolona w konkretnym momencie w czasie;
7. `occupants` - lista klasy `Entity`, będąca reprezentacją aktualnych pasażerów windy.

Interfejs `ElevatorListener`: Wzorzec obserwatora umożliwiający powiadamianie zewnętrznych komponentów (np. GUI) o każdej zmianie stanu windy. Rejestracja przez `addListener`, wyrejestrowanie przez `removeListener`.

3.3.2 Opis stanów `Elevator.State`

W celu określenia aktualnego stanu (stateczny / poruszania się), jak i kierunku poruszania się windy, stworzono podklasę `Elevator.State`. Poniżej znajduje się opis stanów zawartych w ramach tej klasy:

- UP - winda porusza się w górę kondygnacji;
- DOWN - winda porusza się w dół kondygnacji;
- IDLE - winda stoi - nie ma pasażerów, ani osób oczekujących na piętrach.

3.3.3 Kolejka `destinationQueue`

Kolejka `destinationQueue` jest najistotniejszym elementem całej implementacji. W praktyce, jest to lista dwuelementowych list postaci

`[numer_piętra, priorytet_piętra]`. Wartość priorytetu jest zależna od tego, w jaki sposób dane piętro zostało dodane do kolejki:

- 1 - wpis dodany na koniec kolejki, wywołanie przez osobnika z poziomu piętra (funkcja `callElevator`);
- -1 - wpis wstawiony ze średnim priorytetem, wywołane przez osobnika w kabinie jako jego piętro docelowe;
- -2 - wpis aktualnie obsługiwany, wstawiony z największym priorytetem.

Największa potencjalna liczba elementów kolejki `destinationQueue` to liczba wszystkich pięter, z czego maksymalnie `maxCapacity` z nich ma ujemną (czyli priorytetową) wartość priorytetu.

3.3.4 Najważniejsze metody:

`processQueue(Division division)`

Pętla główna symulacji. Funkcja przetwarza kolejkę `destinationQueue` do wyczerpania, i jest wywoływana gdy do pustej kolejki zostanie dodane nowe żądanie. W każdej iteracji funkcji wykonywana jest następująca kolejność instrukcji:

1. Głowę kolejki `destinationQueue`, tj. piętro o priorytecie -2 - czyli to, do którego winda musi dojechać zanim zmieni kierunek / zatrzyma jazdę;
2. Wyznaczany jest `currentState` - kierunek ruchu na podstawie pozycji `currentFloor` względem głowy `destinationQueue`;
3. Dopóki `currentFloor` nie równe jest piętru z głowy `destinationQueue`, winda jest przesuwana o jedno piętro (funkcja `moveOneFloor`) i obsługuje postój (`handleFloorStop`), jeśli bieżące piętro jest **gdziekolwiek** na liście docelowej. Tutaj może wydarzyć się jedna z trzech sytuacji:
 - Do windy wchodzi osoba oczekująca na piętrze - aktualizacja `occupants` windy i `waitingEntities` piętra oraz dodanie `destinationFloor` osoby na koniec elementów z priorytetem -1 (czyli między ostatnim elementem -1 a pierwszym elementem +1, jeżeli taki istnieje);
 - Z windy wychodzi osoba - aktualizacja `occupants` windy i usunięcie `destinationFloor` z `destinationQueue`;
 - Winda jest zapełniona - nie zmienia się stan `occupants` ani `destinationQueue`.

4. Gdy kolejka dojdzie do piętra z głowy `destinationQueue`, **co najmniej jeden pasażer na pewno wysiada**, usunięta jest aktualna głowa i ustalona jest nowa głowa kolejki `destinationQueue`.

`enqueueDestination(int floor)`

Dodaje piętro na koniec kolejki, jeśli nie ma go jeszcze w kolejce i jest różne od aktualnego piętra. Wykonywane w ramach funkcji `callElevator`.

`pushDestinationFront(int floor)`

Wstawia piętro z wysokim priorytetem (przed wszystkimi wpisami bez priorytetu ujemnego). Stosowane, gdy pasażer wsiada i jego piętro docelowe powinno być obsłużone wcześniej.

`handleFloorStop(Division division, int floorNum)`

Obsługuje postój windy: najpierw wysiada pasażerów z bieżącym piętrem docelowym, następnie wsiada oczekujących (o ile nie przekroczono pojemności), dodaje ich piętra docelowe priorytetowo do kolejki i symuluje czas postoju (`IOwaitMs`).

`isFull()`, `isEmpty()`

Sprawdzają, czy kabina jest zapełniona do maksymalnej pojemności lub pusta.

`getDirection()`

Zwraca przewidywany kierunek ruchu (UP/DOWN/IDLE) na podstawie pierwszego wpisu w kolejce.

3.4 Klasa `Division`

Reprezentuje pion budynku - zbiór pięter obsługiwanych przez jedną windę. Łączy obiekty `Floor` z obiektem `Elevator` i udostępnia wysokopoziomowe API do wywoływania windy.

Najważniejsze metody:

`Division(String name, int startFloor, int endFloor, Elevator elevator)`

Konstruktor tworzy listę pięter od `startFloor` do `endFloor` (włącznie). Rzuca `IllegalArgumentException`, gdy `endFloor < startFloor`.

`callElevator(int originFloor, int destinationFloor)`

Główna metoda API: tworzy pasażera na piętrze `originFloor` (przez `Floor.spawnEntity`) i dodaje to piętro do kolejki windy (`Elevator.enqueueDestination`). Zwraca referencję do nowo utworzonego pasażera.

`getFloor(int floorNumber)`

Zwraca obiekt `Floor` dla podanego numeru piętra. Rzuca `IndexOutOfBoundsException` przy numerze spoza zakresu.

`getMinFloor()`, `getMaxFloor()`

Wyznaczają odpowiednio najniższe i najwyższe piętro w pionie, korzystając z naturalnego porządku obiektów `Floor`.

`printStatus()`

Wypisuje na standardowe wyjście aktualny stan wszystkich pięter oraz windy
- pomocnicze narzędzie diagnostyczne.

3.5 Klasa ElevatorGUI

Pakiet: GUI

Główne okno aplikacji zbudowane na bazie `JFrame` (Swing). Odpowiada za wizualizację stanu symulacji w czasie rzeczywistym oraz interakcję z użytkownikiem. Komunikacja z warstwą modelu odbywa się przez interfejs `ElevatorListener` - winda wywołuje `updateUIState()` po każdym ruchu i postoju.

Układ interfejsu:

- **Północ** (`BorderLayout.NORTH`) - panel parametrów symulacji (zakresy pięter, prędkość, czas postoju, pojemność) z przyciskiem *Zastosuj i restartuj*,
- **Centrum** - dwie kolumny: lista pięter (z zaznaczeniem aktualnej pozycji windy) i panel stanu windy (piętro, prędkość, kierunek, pasażerowie, kolejka),
- **Południe** - panel dodawania pasażera ręcznego oraz przyciski sterowania symulacją.

Najważniejsze metody:

`initSimulationFromInputs()`

Odczytuje wartości ze spinnerów, tworzy nowy obiekt `Elevator` i `Division`, rejestruje listener GUI oraz dodaje domyślnych pasażerów startowych.

`startSimulationThread()`

Uruchamia wątek symulacji (jeśli nie jest aktywny), który wywołuje `Elevator.handleFloorStop` oraz `Elevator.processQueue`.

`stopSimulation()`

Ustawia flagę `stopRequested` i przerywa wątek symulacji przez `Thread.interrupt()`.

`spawnRandomPassenger()`

Losuje parę pięter (różnych) w ramach aktualnego zakresu `Division` i wywołuje `callElevator`. Jeśli winda stoi i symulacja jest aktywna, automatycznie wznowia wątek.

`addManualPassenger()`

Pobiera wartości ze spinnerów *Z piętra* i *Na piętro* i wywołuje `callElevator`. Waliduje, że piętra są różne.

`updateUIState()`

Aktualizuje wszystkie etykiety i listę pięter za pomocą `SwingUtilities.invokeLater`, zapewniając wywołanie na wątku EDT.

4 Uruchomienie projektu

4.1 Wymagania

- **Java Development Kit (JDK)** w wersji 17 lub nowszej.
- System operacyjny z obsługą Java Swing (Windows, Linux z X11/Wayland, macOS).
- Opcjonalnie: **Maven** lub **Gradle** do zarządzania budowaniem projektu (projekt nie posiada pliku `pom.xml` ani `build.gradle` - poniżej podano ręczną kompilację).

4.2 Kompilacja i uruchomienie z wiersza poleceń

Zakładając strukturę katalogów zgodną z archiwum `src.zip`:

```
1 cd src/main/java
2 javac -d out elevator/model/*.java GUI/ElevatorGUI.java
3 java -cp out GUI.ElevatorGUI
```

Listing 1: Kompilacja projektu

4.3 Uruchomienie z IDE

1. Otwórz projekt w wybranym środowisku (IntelliJ IDEA, Eclipse, NetBeans).
2. Ustaw katalog `src/main/java` jako *source root*.
3. Uruchom metodę `main` w klasie `GUI.ElevatorGUI`.

4.4 Konfiguracja parametrów w GUI

Po uruchomieniu aplikacji dostępny jest panel *Parametry symulacji* pozwalający na zmianę:

Parametr	Domyślna wartość	Opis
Najniższe piętro	-1	Minimalna kondygnacja pionu
Najwyższe piętro	7	Maksymalna kondygnacja pionu
Piętro startowe	4	Pozycja windy na starcie
Pojemność windy	5	Maksymalna liczba pasażerów w kabinie
Czas przejazdu (ms)	1000	Czas podróży między sąsiednimi piętrami
Czas postoju (ms)	3000	Czas oczekiwania przy wsiadaniu/wysiadaniu

Kliknięcie przycisku *Zastosuj i restartuj* zatrzymuje bieżącą symulację, reinicjalizuje model z nowymi parametrami i dodaje domyślnych pasażerów startowych.

5 Architektura i przepływ danych

Projekt stosuje uproszczoną architekturę warstwową:

1. **Model** (`elevator.model`) - cała logika symulacji, niezależna od interfejsu.
2. **Widok/Kontroler** (GUI) - interfejs Swing obsługujący zdarzenia użytkownika i odświeżający widok przez `ElevatorListener`.

Przepływ typowego zdarzenia *wywołanie windy*:

1. Użytkownik klika *Dodaj* lub *Dodaj losowego pasażera* w `ElevatorGUI`.
2. `ElevatorGUI` wywołuje `Division.callElevator(from, to)`.
3. `Division` tworzy `Entity` na odpowiednim `Floor` i dodaje piętro do kolejki `Elevator`.
4. Wątek symulacji (uruchomiony przez `startSimulationThread`) przetwarza kolejkę przez `Elevator.processQueue`.
5. Po każdym ruchu i postoju `Elevator` wywołuje `notifyListeners()`, co uruchamia `ElevatorGUI.updateUIState()` na wątku EDT.

6 Podsumowanie

[Miejsce na podsumowanie - do uzupełnienia przez autora.]